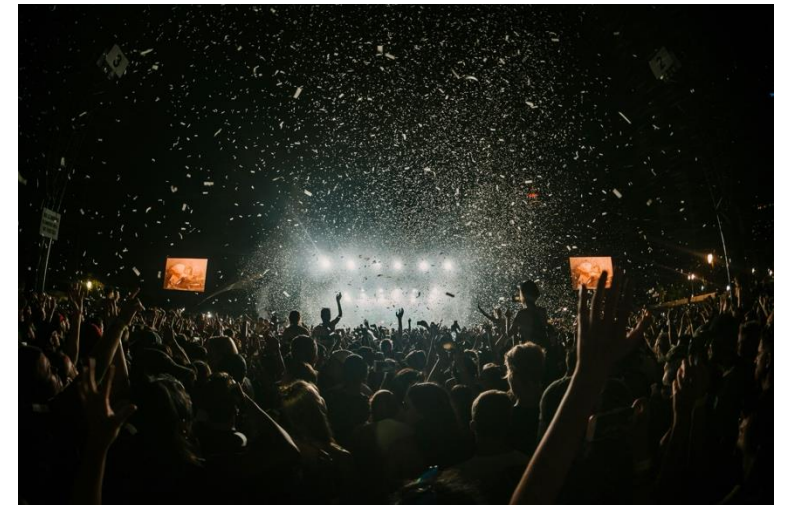
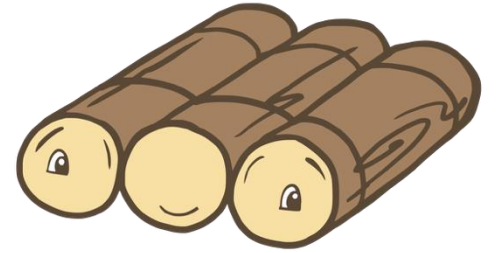


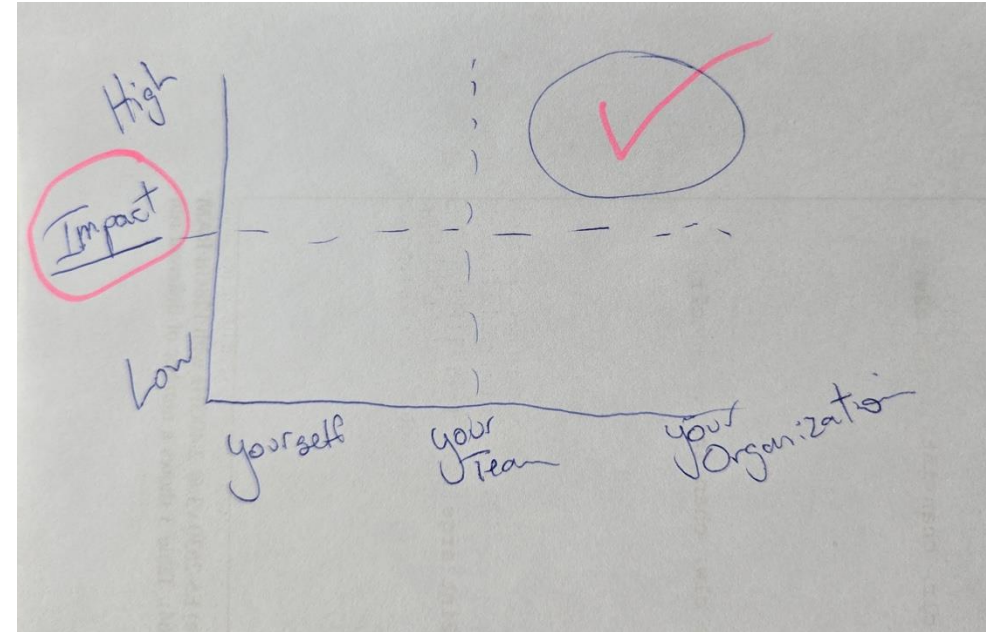
Week 5: Wait a minute... what are the *advantages* and *disadvantages* of a distributed system?

AND WHAT DOES THIS HAVE TO DO WITH US?!?!



Centralized versus Distributed Systems

- Pros/cons of each?
- Is this limited to *computer* systems?!
- How about *our* communication patterns?
 - You are great students! Let's get GREATER!
 - How could we be scaling assistance better on piazza?
 - Is this of value in your future careers at all?



How do we move you into that TOP RIGHT!?!?

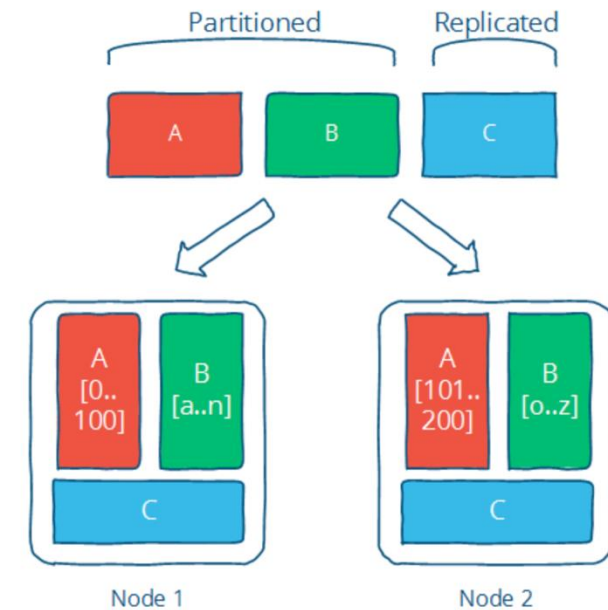
- EXAMPLE (REAL WORLD!) SCENARIO
 - ***Work with your group on this!***
 - Deploying software on the cloud
 - Containers are *crashing*...
 - O my.... Not good! What's going on?!
 - Jr. Devs meeting with bosses
 - If this was you, how do you prepare for this meeting?!
- So, what do we do now that we know this?!
 - HOLD THAT THOUGHT!
 - First a quick review!

Distributed Systems for Fun and Profit! Chpt 1

- Storage and computation in a data center
 - *“Scalability is the ability of a system to handle a growing amount of work in a capable manner **or** its ability to be enlarged to accommodate that growth!”*
- meets the needs of its users in terms of
 - Performance
 - Short response time/low latency
 - High throughput (rate of processing work)
 - Low utilization of computing resource(s)
 - Availability
 - systems with redundancy can be tolerant of partial failures and thus be more available
 - “fault tolerant”

Distributed Systems for Fun and Profit! Chpt 1

- Abstractions and Models
 - System model (asynchronous / synchronous)
 - Failure model (crash-fail, partitions, Byzantine)
 - Consistency model (strong, eventual)
- Design Techniques for Scalability
 - Partitioned
 - Replicated



Google Tutorial

- "8 Fallacies" that can GET YOU! 😊
 - The network is reliable.
 - Latency is zero.
 - Bandwidth is infinite.
 - The network is secure.
 - Topology doesn't change.
 - There is one administrator.
 - Transport cost is zero.
 - The network is homogeneous.
- **IMPORTANT!** Be sensitive to performance!
 - Where are the bottlenecks and why?
 - *Devise small tests you can do to evaluate alternatives...*
 - Profile and measure to learn more!
 - *Talk to your colleagues about these alternatives and your results, and decide on the best solution!*

The MOST important “skill” for your future!

- Meetings in groups for *this* reason
 - Compare and contrast Homework results!
 - Build/test/build/test... (from Google Tutorial, *ALTERNATIVES!*)
- Eventually...
 - Group REPORT!
 - Presentations!
 - Sharing reports with other groups!

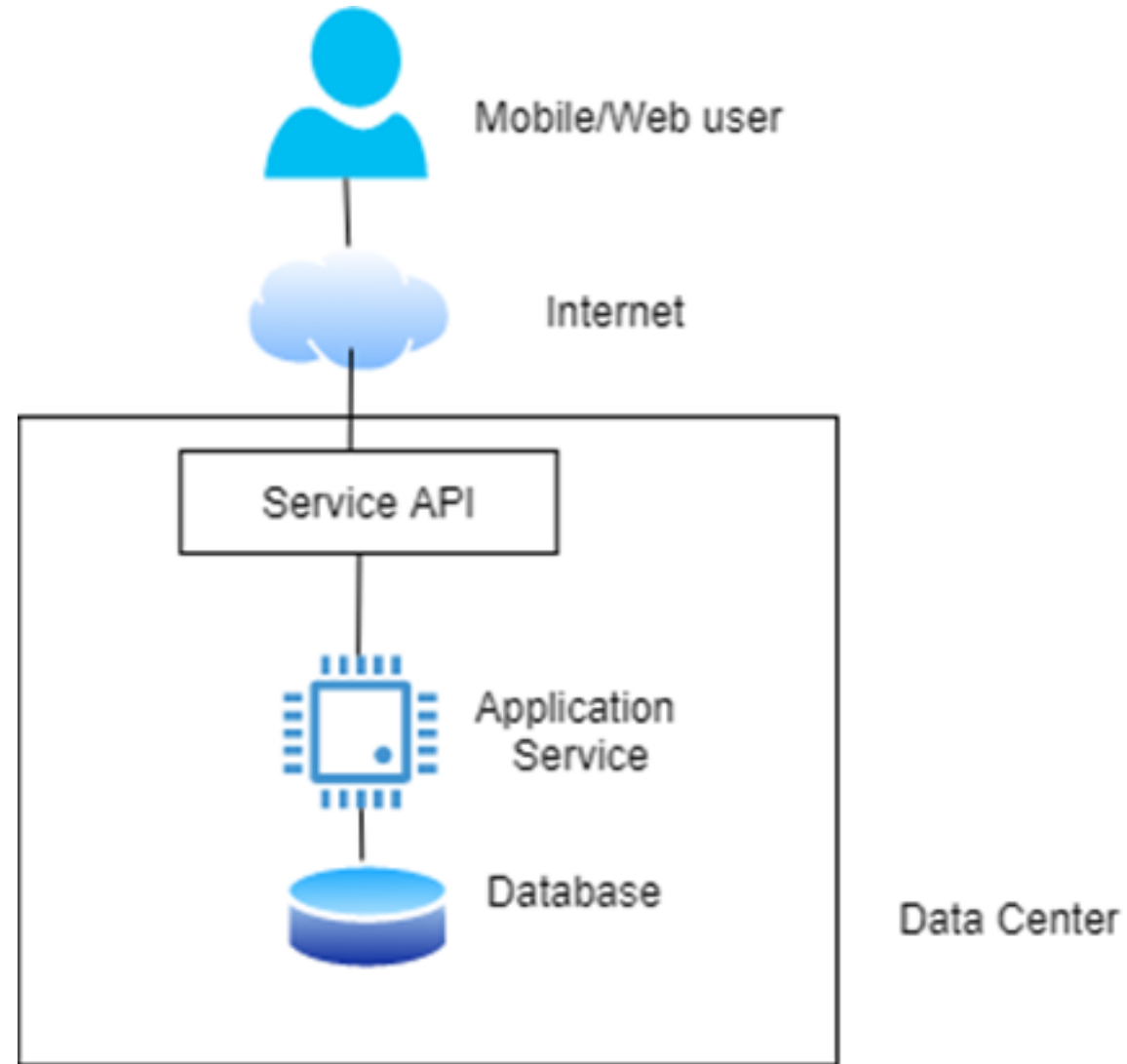
IDEA! What do you think?!

- Demonstrate your ability to trouble shoot in your Mock Interviews!
 - Something to bring to the group, 3 stages of problem solving...
 1. What problem you encountered, and what you tried to do to solve it (stage 1!)
 2. What you encountered when you tried your idea (in stage 1), and what you tried next...
 3. What you encountered when you tried your next idea (in stage 2), and **then** your articulation of the problem, and what you THINK is going on!
 - ALSO, show a reply to a question/comment YOU posted on piazza!
 - Note that helping others is a BIG way of showing value within your organization!
- Can you see how this is different than what is going on right now?!

Week 5

- Topics we will cover today
 - Service API Design
 - CRUD Pattern
 - OpenAPI/Swagger
 - Application Services Containers
 - Horizontal Scaling
 - Load Balancing

Service API



Service API Design

- An API defines a **contract** between the client and server
 - If you send me *this* data, then I will promise to do *this*
- Dominant style is HTTP APIs
- **CRUD Pattern - Create, Read, Update, Delete**
 - Almost every key object has those four methods
- REST is one specific style of Service API
 - REST == **RE**presentational **St**ate **T**ransfer
 - “REST” is often used when it should be the more general “Service”
 - Useful for *CRUD*!

REST-style vs RPC-style

- Every Domain Object (noun) has a url with an ID
 - `../shoppingcart/123456`
- HTTP methods: GET, POST, PUT, DELETE
 - Send a complete object using one of those four methods
 - To add an item to a shopping cart, some choices
 - ShoppingCart contains a list of items, send PUT and server has to work out that you are adding a new item
 - *Note that PUT is for "here is the exact resource at this location."*
 - Item is a separate noun, contains a Shopping Cart ID and send an item-POST
 - *Note that POST is for "here is some data; please create a new resource for it."*
- No verb operations, server has to untangle the meaning
 - No `shoppingCart.addItem()`

RPC-Style

- RESTful fails the “Say What You Mean” test
 - API is harder to understand
 - Server has extra work out to decide what the client wanted
- For example, with RPC it would be
 - `createShoppingCart()`
 - `addItem()`
- Therefore we use RPC *style* APIs, not REST

CRUD HTTP APIs

Verb	Example URL	Purpose
POST	<code>/shop.com/register/</code>	Create a new customer profile with customer details provided in the JSON request payload. The newly created profile is returned in the JSON response payload.
GET	<code>/shop.com/customer/{customerID}</code>	Get the profile information for a customerID, returned in a JSON response payload
PUT	<code>/shop.com/customer/{customerID}</code>	Update customerID profile
DELETE	<code>/shop.com/customer/{customerID}</code>	Delete a customerID profile because they asked us to

CRUD HTTP APIs

- A HTTP CRUD API applies HTTP methods on **resources** identified by Uniform Resource Identifiers (URIs)
`/shop.com/customer/768934`
- The **HTTP method and URI** define the semantics of the API operation
- Payloads typically formatted as JSON
 - XML or text also possible
- Resources, represented by URIs, are like objects in Object Oriented Design (OOD)

OpenAPI and Swagger

OpenAPI

- Commonly used specification language for HTTP APIs
- Originally known as **Swagger** Specification
- Swagger is a set of tools for OpenAPI
 - Editors, parsers, code generators
 - Swagger.io



API Specification – OpenAPI

```
57 /products/{productId}/details:
58   post:
59     tags:
60       - Products
61     summary: Add product details
62     description: Add or update detailed information for a specific product
63     operationId: addProductDetails
64     parameters:
65       - name: productId
66         in: path
67         required: true
68         description: Unique identifier for the product
69         schema:
70           type: integer
71           format: int32
72           minimum: 1
73     requestBody:
74       required: true
75       content:
76         application/json:
77           schema:
78             $ref: '#/components/schemas/Product'
79     responses:
```



```
419 components:
420   schemas:
421     Product:
422       type: object
423       required:
424         - product_id
425         - sku
426         - manufacturer
427         - category_id
428         - weight
429         - some_other_id
430       properties:
431         product_id:
432           type: integer
433           format: int32
434           minimum: 1
435           description: Unique identifier for the product
436           example: 12345
437         sku:
438           type: string
439           minLength: 1
440           maxLength: 100
441           description: Stock Keeping Unit - unique product code
442           example: "ABC-123-XYZ"
```

Designing Services

- Application server requests to the appropriate handler function
- Handler defined by the application and implements the business logic
- Each request is (typically) **allocated a thread** to execute the request
- Example Express.js server

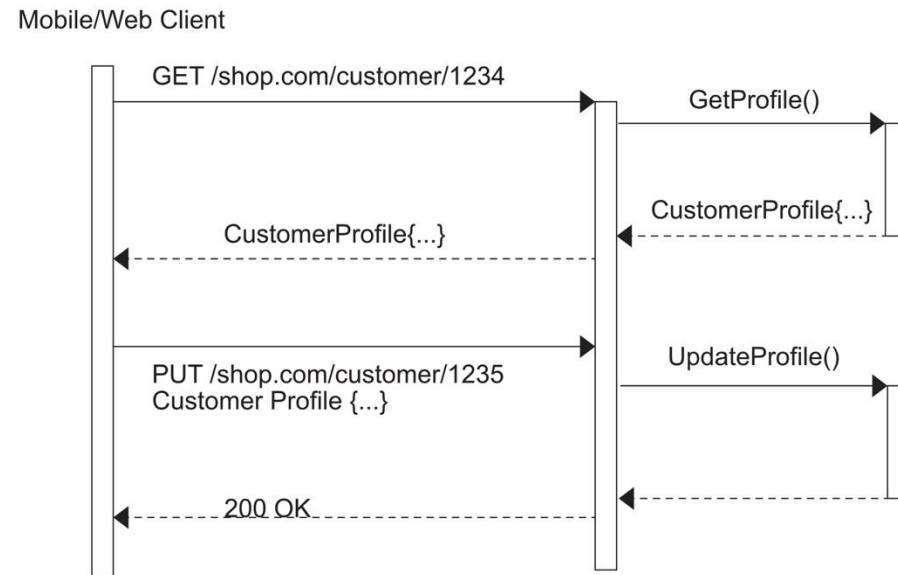
```
app.get('/customer/:customerID', function (req, res) {  
    // process the GET request  
    ProcessRequest(req.params)  
})
```

Problems with Stateful Services

- Session state uses available server memory
- How long to keep session state available?
- Load balancing is difficult
 - Either all messages in a conversation must go to same machine, or
 - Services need a look-aside cache for session state
- Not resilient
 - When a server crashes all those conversations are lost!!!

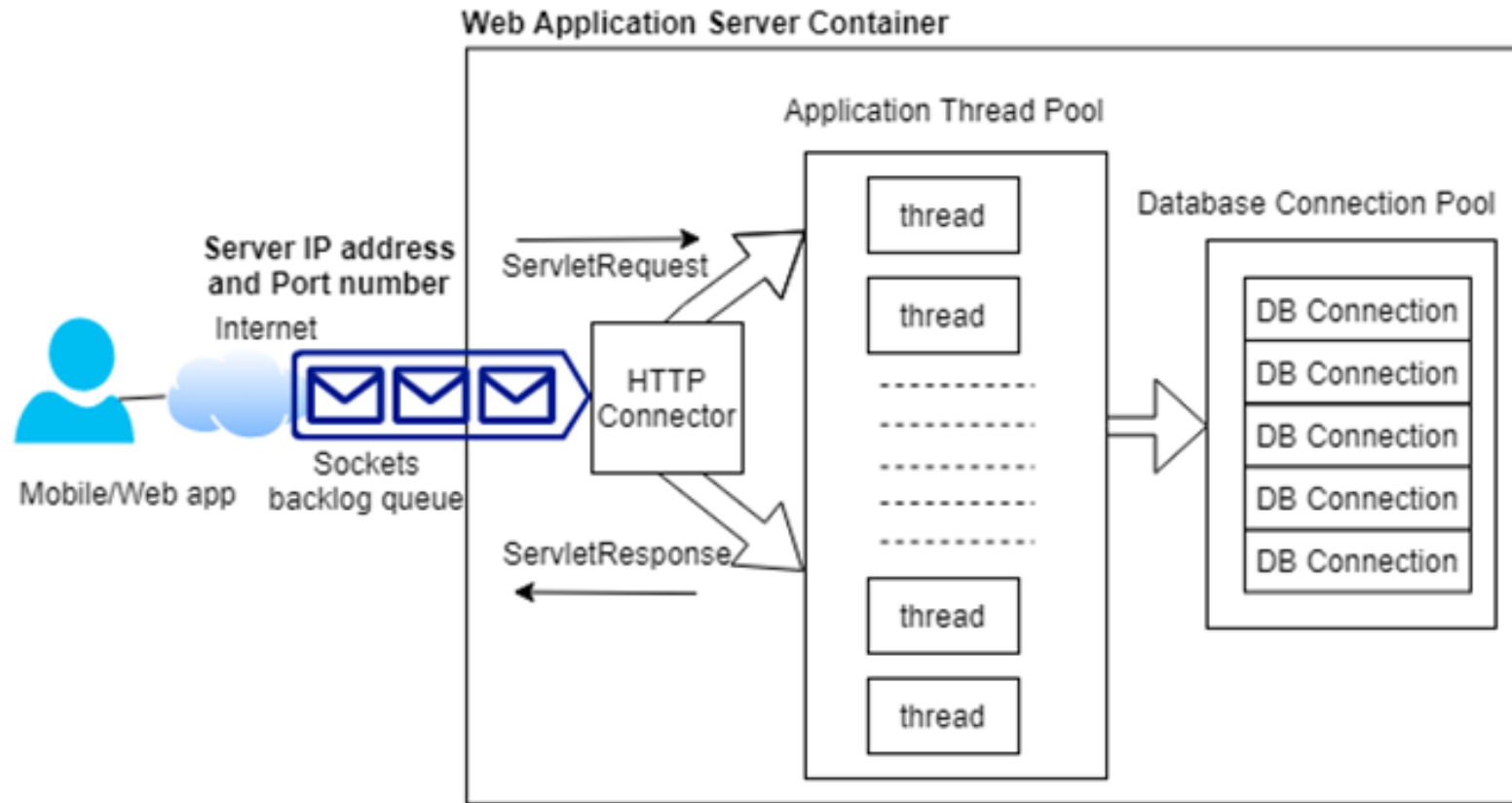
Stateless Services

- Stateless services **do not store** conversational state
- Each request is standalone and can be processed individually
- Client must provide all the necessary information to the
 - PUT /shop.com/customer/768934/phoneno/4123131169



Application Servers

- Host the business services that comprise an application
- Provide necessary utilities
 - Database connectivity
 - Logging
 - Threading
 - A very long list...
- Many application servers
 - Spring Boot
 - Java Enterprise Edition (JEE)
 - Express.js for Node
 - Flask for Python
 - GoLang net/http package
 - Note that Gin is a web framework for Go
 - builds upon net/http to offer more



Tomcat Example

Performance Tuning

- Tuning configuration parameters is important
 - If CPU utilization consistently exceeds 70-80% => overload!
 - Similar thresholds exist for memory usage
- When any resource approaches full utilization, performance is unpredictable
 - Similar to a hash table that is close to full, performance *suddenly* degrades
- Performance is always tricky
 - Requires a load-test harness
 - Default settings are rarely what you need
 - Careful experimentation paying attention to detail

Monitoring & Alarm Tools

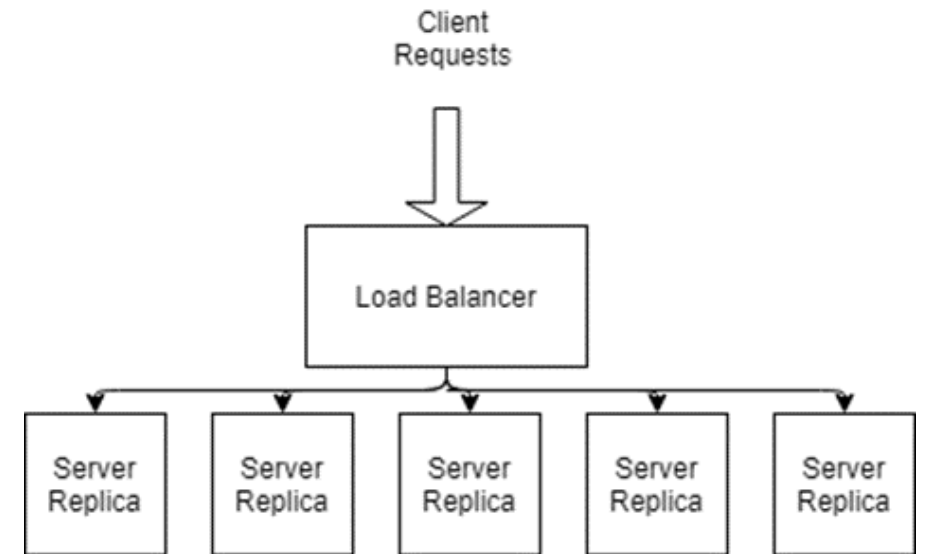
- Monitoring tools exist for Web application frameworks
 - Latencies
 - Active requests
 - Queue sizes
- Many monitoring systems
 - Grafana
 - Elastic Search/Kibana

Grafana



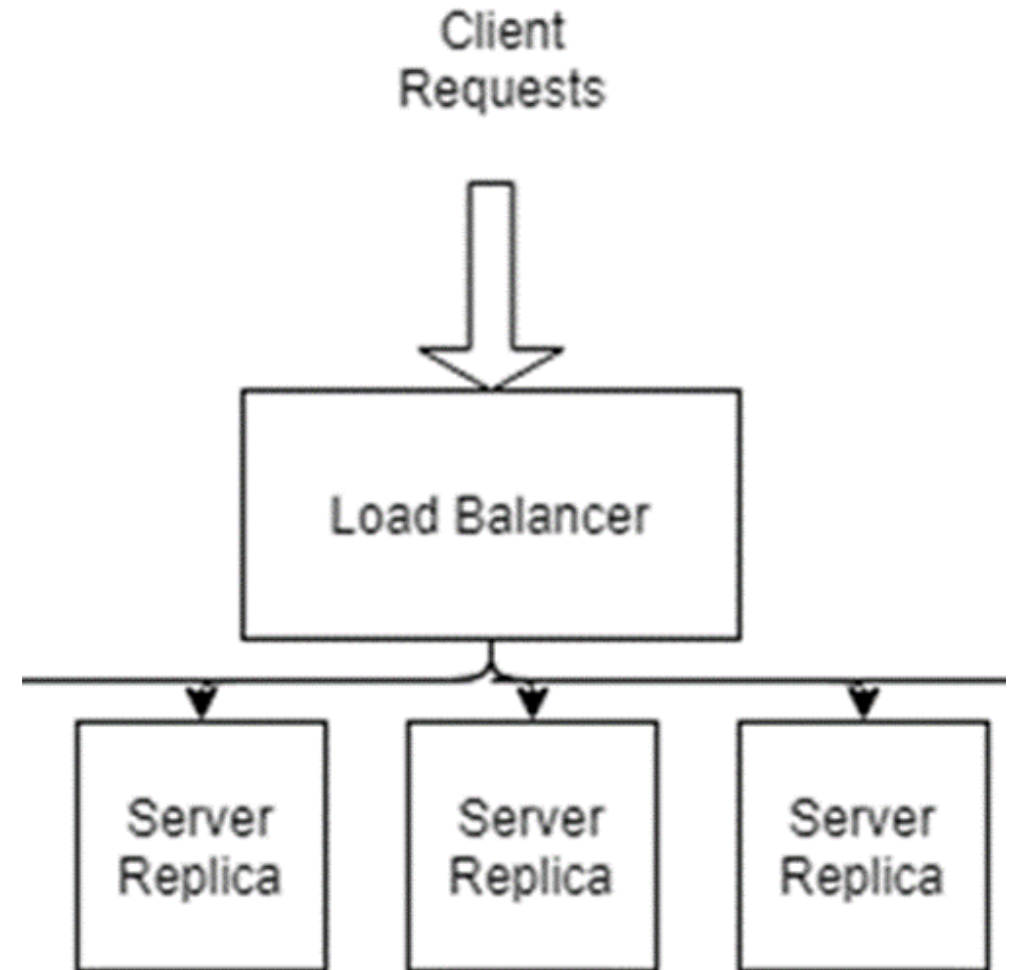
Horizontal Scaling

- Service replicas deployed on their own (virtual) hardware
- Services must be **stateless**
- *Any* request sent to *any* service by load balancer
- Horizontal scaling also increases availability
 - Eliminates single point of failure (SPoF)
 - If a replica fails, requests re-directed to any replica



Load Balancing

- *Aims* to utilize capacity of a collection of services to optimize response time for each request
- Distributes requests across the available services as evenly as possible
- Clients send requests to the load balancer
 - Requests redirected to target services
 - Results relayed back to the client



AWS Icons

- distributes incoming application traffic across multiple targets
- enhances application availability, scalability, and fault tolerance



Elastic Load
Balancing*

Elastic Load Balancing automatically distributes your incoming application traffic across multiple targets, such as EC2 instances. It monitors the health of registered targets and routes traffic only to the healthy targets.



Classic Load
Balancing

A **classic load balancer** distributes incoming application traffic across multiple EC2 instances in multiple Availability Zones. This increases the fault tolerance of your applications. Elastic Load Balancing detects unhealthy instances and routes traffic to only healthy instances.

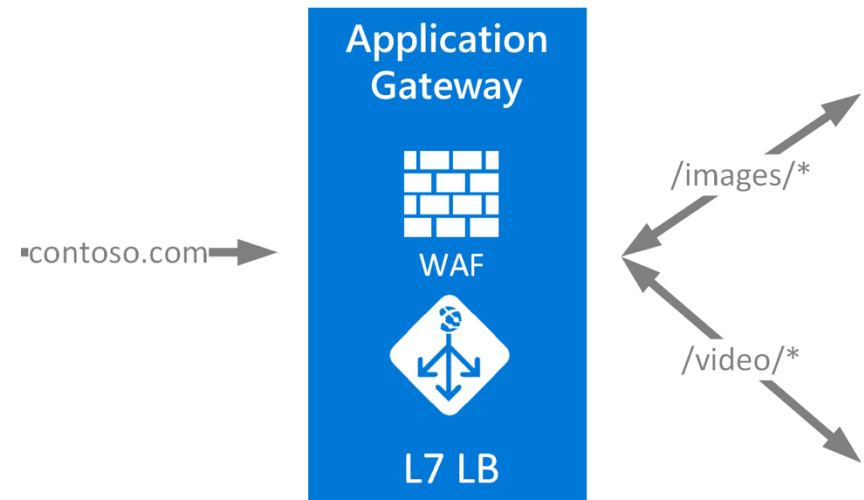


Application
Load
Balancing

An **application load balancer** serves as the single point of contact for clients. The load balancer distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. This increases the availability of your application. You can add one or more listeners to your load balancer.

Application Load Balancers

- Routing decisions based on HTTP headers and the contents of the message
 - A WAF (Web Application Firewall) filters, monitors, and blocks HTTP/HTTPS traffic
- For example
 - Send all POST requests to a subset of available services
 - Distribute requests based on a query string in the URI



3 Types of ELASTIC Load Balancing

- **Application Load Balancer (ALB):**

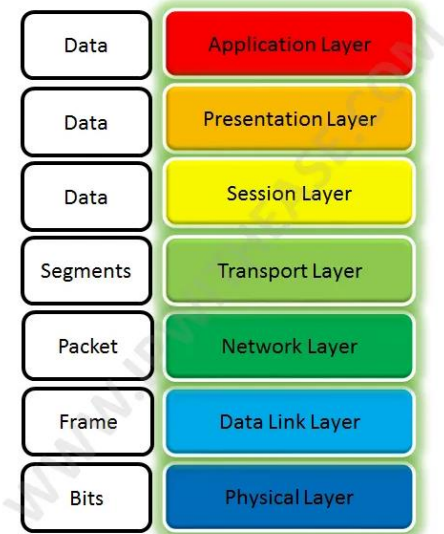
- operates at the application layer (Layer 7 of the OSI model)
- ideal for HTTP and HTTPS traffic
- supports containerized applications and integrates with services like Amazon EC2, Amazon S3, and Amazon ElastiCache

- **Network Load Balancer (NLB):**

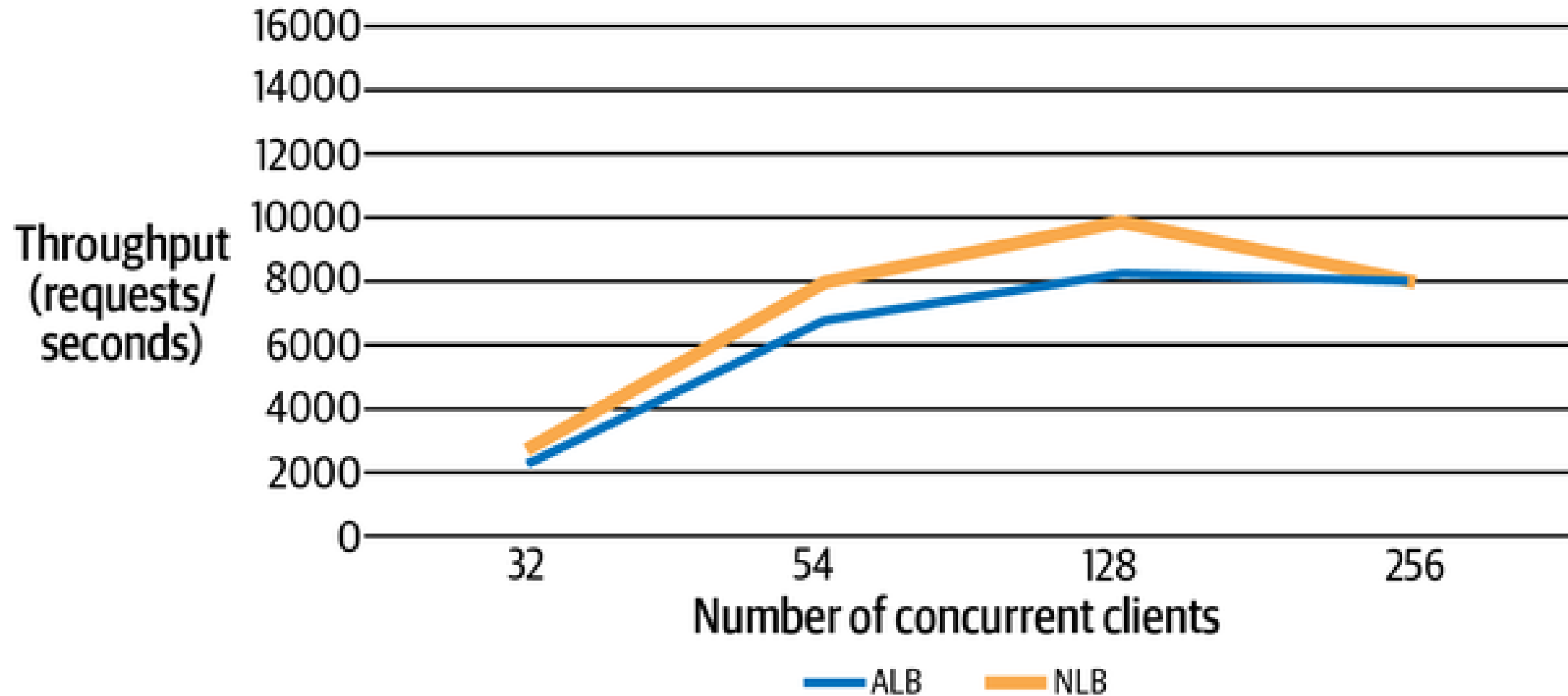
- operates at the transport layer (Layer 4 of the OSI model)
- designed for high-performance and low-latency scenarios, millions of requests per second for TCP and UDP traffic

- **Gateway Load Balancer (GWLB):**

- operates at the network layer (Layer 3 of the OSI model)
- manages firewalls, intrusion detection/prevention systems, and other security solutions



Performance Comparison



Load Balancer Policies

- Round-robin
- Fewest connections
- Server load
- HTTP header field
- HTTP operation
- Server weights
-



Load distribution
policies



Health
monitoring



Elasticity



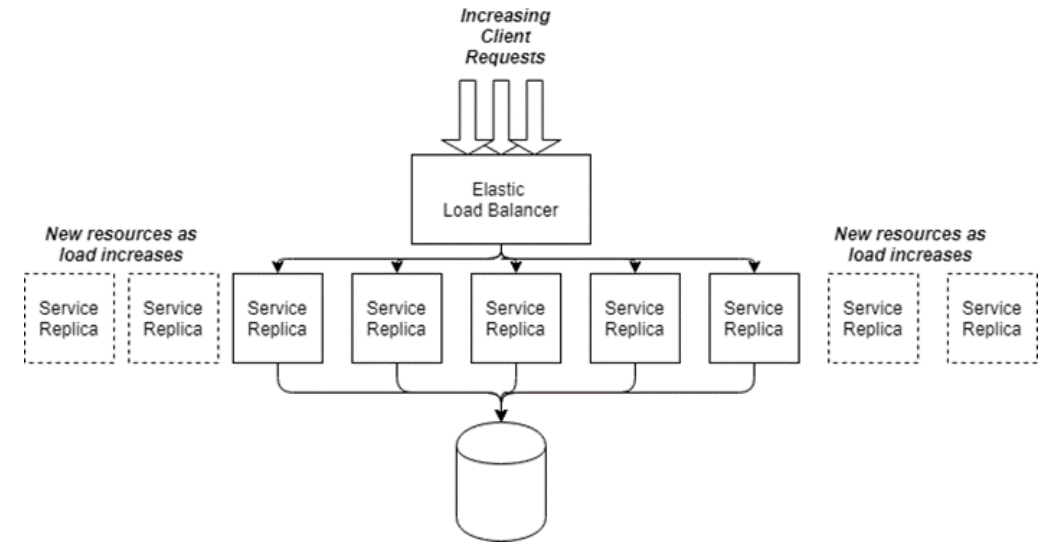
Session affinity

Health Monitoring

- Periodically *ping* or attempt connections to test health of each service in load balancing pool – Health Check
- Remove a server if it
 - Is unresponsive (does not answer the ping)
 - Fails connection attempts from client
- Load-based balancing originally required periodic pings to sample the server load
 - Some LBs now monitor error rates and adjust accordingly

Elastic Scaling

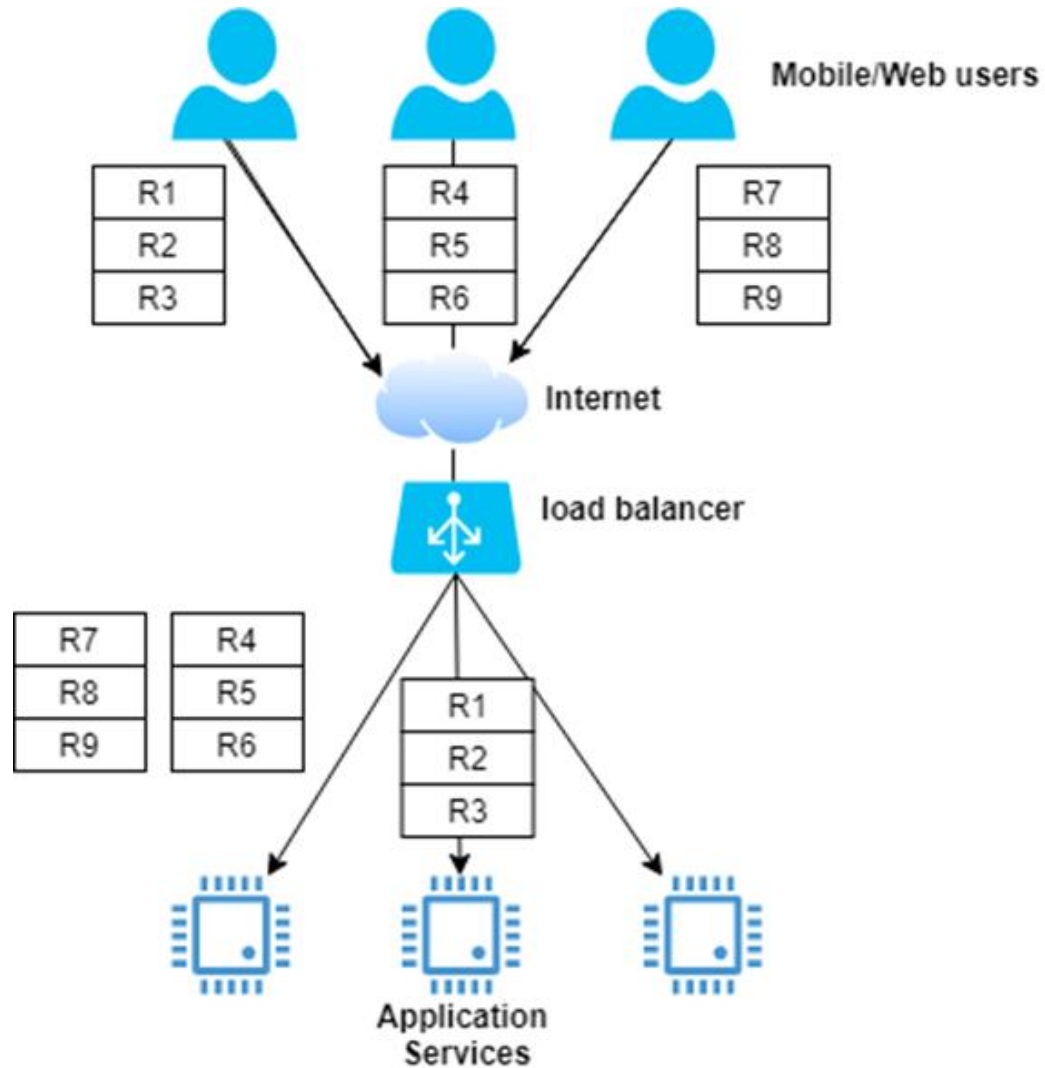
- Two ways to control the number of services in a group
 - Schedule-based (black Friday?!)
 - Dynamic by CPU usage or memory or ...
- Instances need time to startup
 - Not available until after warmup period
 - Booting, starting applications, loading caches
- Scale down when average CPU utilization drops below threshold



Session Affinity (Sticky Sessions)

- Load balancer sends all requests from the **same** client to the **same** service instance
- AWS Elastic Load Balancing generates an HTTP **cookie** that identifies the service that a client's session is associated with
- Cookie is returned to client, which must send it in subsequent request to ensure session affinity is maintained
 - Requires a session database
 - More complicated

Load Imbalance (for Sticky Sessions)



Stateful versus Stateless

- **Stateful** services have other downsides
 - How do clients connect when their sticky instance fails?
 - How do clients respond when their instance is slow?
- **Stateless** services do not have these problems
 - If service fails, the clients retries and request routed to another instance
 - If a service is slow, the load balancer takes it out of the service group until it passes health checks
- **Stateless** services
 - Enhance scalability
 - Simplify failure scenarios
 - Ease the burden of service management

Scalable Service Design - Summary

- Service API specifies contract with client
- Application service containers provide powerful multithreaded execution environments
- Performance tuning and monitoring are crucial for optimization
- Load balancing with stateless services is a key scalability pattern

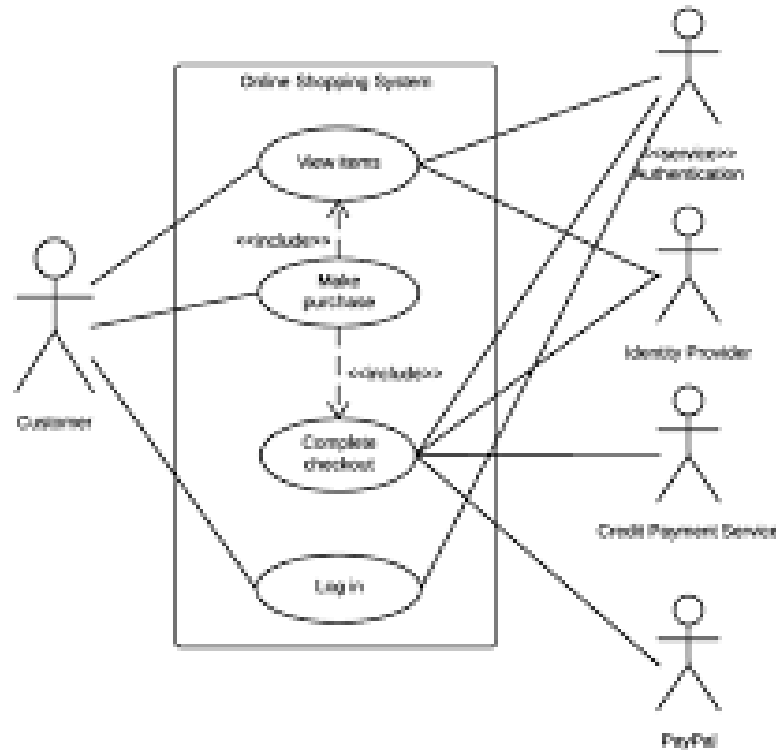
Use Cases!

Why did someone pay me to build this system?

- As an engineer you want to know a system's purpose
 - Especially now, with AI-generated code!!!!
- Use cases are a great method for requirements
- Other methods
 - Stories in Agile. Similar to use cases but less formal, therefore can be ambiguous
 - Long lists of “the system shall do X”
 - No coherent vision
 - No framework that helps you spot holes
 - No meaning to the actual users

Use Cases are text, not stick figure diagrams

- Use Cases diagrams are the worst part of UML
 - Fiddly to draw
 - No useful information



Actors & Domain Objects

- Actor
 - Customer, Scanner, Truck, Clock
 - Something external to the system that triggers system activity
 - Really should be “Role” but the word “Actor” is traditionally used
- Domain Object
 - Credit Card
 - Shopping Cart
 - Product
 - Important nouns in use cases with precise meanings
 - Almost always exist in the software system

Why Use Cases in This Course?

- You must perform realistic load testing
- That means you should be able to answer the question...
 - How is your system being used by your customers?

ATM Example – Who can do What with an ATM?

- Customer ...

Use Case Titles – 90% of the time

- Title syntax: <Actor> Verb <Domain-Object>
 - Customer Withdraws Cash
 - Customer Adds Item to Shopping Cart
 - Customer Checkouts Shopping Cart
 - Customer Shows Boarding Pass
 - Customer Claims Bag
 - Customer Complains to Manager
 - Truck Driver Delivers Package to Customer-Address

Steps – “Sunny Day” Scenario

- Write the simple interaction, when everything works
- Errors and Exceptions
 - Complex optional choices
 - Errors made by the actor (wrong PIN)
 - Problems created by other things
 - No stock in warehouse
 - Network goes down
 - Airline loses luggage

CRUD

- Most Domain Objects have Uses Cases for
 - Create
 - Read
 - Update
 - Delete